Project 1: Genetic Algorithm

(B252-E162110) Soft Computing Techniques to Characterize Thermal Systems

Šimon Hykl[1]
Czech Technical University in Prague

This report covers the genetic algorithm developed in the 'Soft Computing Techniques to Characterize Thermal Systems' class. The purpose of this project is to find global extremes of synthetic multivariable functions, but the algorithm itself is general enough to be used for other purposes. The MATLAB-based script is briefly described and results are shown for two target functions. Effects of the algorithm parameters are studied with focus on the number of generations needed for convergence and the precision of the result.

I. Introduction

The goal of this project is to develop a genetic algorithm able to optimize certain problems by finding global extremes. The functionality will be tested on two synthetic functions.

A. Function of a single variable

The single-variable function is defined as

$$f(x) = \frac{\sin^2(10x)}{x+1}. \tag{1}$$

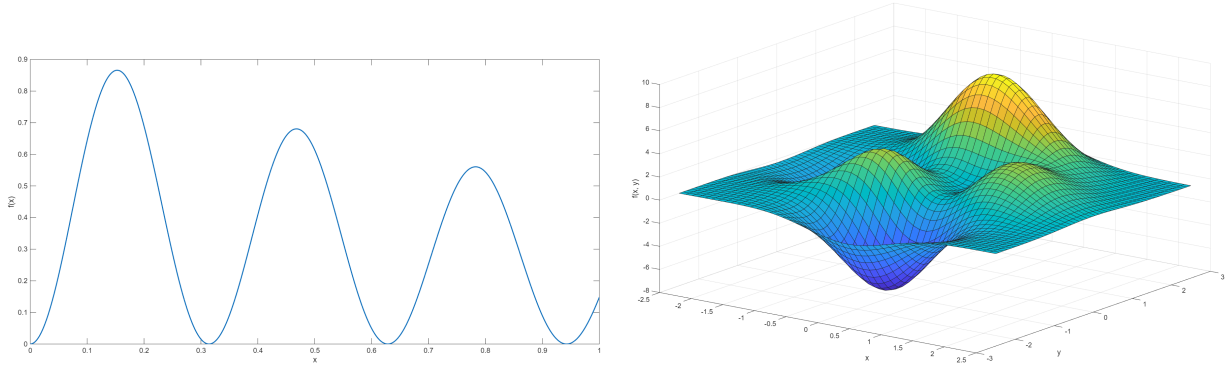
It has a single global maximum $f(x)_{\max} \approx 0.8659$ and infinitely many global minima $f(x)_{\min} = 0$.

B. Function of multiple variables

The second function is defined as

$$f(x,y) = 3(1-x)^2\, e^{-x^2-(y+1)^2} - 10\left(\frac{x}{5} - x^3 - y^5\right) e^{-x^2-y^2} - \frac{1}{3} e^{-(x+1)^2-y^2}. \tag{2}$$

It has a single global maximum $f(x)_{\max} \approx 8.1062$ and a single global minimum $f(x)_{\min} \approx -6.5511$.



(a) 1D **(b) 2D**

Fig. 1 Target Functions

[1]Student at Department of Aerospace Engineering, simon.hykl@fs.cvut.cz

II. Developed Algorithm

At its core, the algorithm repeats a main loop in which the evolution itself happens. This loop runs until the maximum number of generations or until a convergence of the specified order is reached.

For code legibility, the main features of the algorithm are offloaded into separate functions which are called from the main loop. The main loop is nested into two other loops which serve as means to automatize the parameter sweeps and obtain statistical data. Many of the parameters are customizable. Different evolution parameters and plotting settings can be set in the preamble.

Note: The code snippets shown here are only illustrative and are simplified for the sake of legibility.

A. Main loop

Before the loop itself, the initial population of size N is generated randomly between the bounds $x(1)$ and $x(\text{end})$[1] and evaluated.

During the main loop, the conditions for crossover and mutation are checked against their probabilities P_c and P_m and executed accordingly. The new population is created this way. For the new population, an expected result Y_e is estimated as the average of the individuals' function values.

If the convergence functionality is enabled, the loop ends when the residuum of the expected result reaches a desired error order. Otherwise the loop continues until the maximum number of generations is reached.

```
X = x(1) + rand(N, 1).*(x(end) - x(1));
Y = y(1) + rand(N, 1).*(y(end) - y(1));

Z = f(X, Y);

for i = 1:imax
    if rand() < Pc
        [X, Y, Z] = eventCrossover();
    end

    if rand() < Pm
        [X, Y, Z] = eventMutation();
    end

    Ye = mean(Z);

    res = abs(Ye - target);

    if checkConvergence && res < error_margin
        break
    end
end
```

[1] The same applies for y

B. Crossover and Mutation

The crossover function selects two parents and two death candidates form the population. High fitness is favored in the parent selection while the opposite is true for the death candidates. Children created from the parents then replace the death candidates. This way, the most fit individuals should reproduce while the least fit ones should die out. The crossover position is selected randomly.

For the selection process itself, a roulette selection and a tournament selection are implemented.

The function has an integrated check against parent and death candidate duplicity.

The crossover itself works by combining the parents' DNA to form the children.

```
function [X, Y, Z] = eventCrossover()
    while(1)
        parent_index = selectionRoulette(fitness)

        if parent_index(1) ~= parent_index(2)
            break
        end
    end

    while(1)
        death_index = selectionTournament(1 / fitness)

        if parent_index(1) ~= parent_index(2)
            break
        end
    end

    crossover_pos = randi(chrlen - 1, 1, 2);
    n = chrlen - crossover_pos;

    parent_dna_X = dnaConv(X(parent_index));
    parent_dna_Y = dnaConv(Y(parent_index));

    child_dna_X = [parent_dna_X(1, 1:n(1)),
                   parent_dna_X(2, (n(1) + 1):end);
                   parent_dna_X(2, 1:n(1)),
                   parent_dna_X(1, (n(1) + 1):end)];
    child_dna_Y = [parent_dna_Y(1, 1:n(2)),
                   parent_dna_Y(2, (n(2) + 1):end);
                   parent_dna_Y(2, 1:n(2)),
                   parent_dna_Y(1, (n(2) + 1):end)];

    children_X = dnaConv(child_dna_X, 'inverse');
    children_Y = dnaConv(child_dna_Y, 'inverse');

    X(death_index) = children_X;
    Y(death_index) = children_Y;

    Z(death_index) = f(X(death_index), Y(death_index));
end
```

The mutation function works similarly to the crossover. However, there is no selection process and the mutated individual is chosen randomly. The position of the mutated chromozome is also random.

The mutation function uses a different approach to changing the DNA and thus needs to use an alternate version of the DNA conversion function (the 'int' argument must be included).

```
function [X, Y, Z] = eventMutation()
    mutation_index = randi(N, 1, 1);

    mutation_pos = randi(chrlen - 1, 1, 2);

    mask_X = uint64(bitshift(1, mutation_pos(1) - 1));
    mask_Y = uint64(bitshift(1, mutation_pos(2) - 1));

    premutation_dna_X = dnaConv(X(mutation_index), 'int');
    premutation_dna_Y = dnaConv(Y(mutation_index), 'int');

    mutation_dna_X = bitxor(premutation_dna_X, mask_X);
    mutation_dna_Y = bitxor(premutation_dna_Y, mask_Y);

    mutation_X = dnaConv(mutation_dna_X, 'inverse', 'int');
    mutation_Y = dnaConv(mutation_dna_Y, 'inverse', 'int');

    X(mutation_index) = mutation_X;
    Y(mutation_index) = mutation_Y;

    Z(mutation_index) = f(X(mutation_index), Y(mutation_index));
end
```

C. Selection strategies

For the roulette strategy, the probability of selection is weighted by the individual's fitness. In the tournament strategy, a set of participants are selected at random and then the two most fit individuals are chosen.

```
function index = selectionRoulette(fitness)
    probability = fitness / sum(fitness);

    distribution = cumsum(probability);

    parents = rand(1, 2);

    index(1) = find(parents(1) < distribution, 1, 'first');
    index(2) = find(parents(2) < distribution, 1, 'first');
end

function index = selectionTournament(fitness, tpoolsz)
    i_cand = randperm(numel(fitness), tpoolsz);

    cand_fit = fitness(i_cand);

    [~, i_sort] = sort(cand_fit, 'descend');

    index = i_cand(i_sort(1:2));
end
```

D. DNA Conversion

The crossover and mutation functions utilize binary or array operations on the population's DNA. This requires converting the DNA into binary and back to decimal for easy visualization.

The default conversion direction is decimal to binary. For the inverse conversion, the 'inverse' argument must be appended.

As was mentioned before, the mutation function requires the binary values to be represented as unsigned integers. For this purpose, the 'int' argument must be appended.

```
function dna = dnaConv()
    binbase0 = flip(2^(0:1:(chrlen - 1)));
    binbase = ones(size(data, 1), 1)*binbase0;

    maxval = 2^chrlen - 1;

    if strcmp(direction, 'inverse')
        if strcmp(dattype, 'int')
            data = double(dec2bin(data, chrlen)) - '0';
        end

        x01 = sum(data*binbase, 2) / maxval;

        dna = range(2)*x01 + range(1)*(1 - x01);
    else
        x01 = (data - range(1)) / abs(range(2) - range(1));

        x = x01*maxval;

        dna = mod(floor(x / binbase), 2);

        if strcmp(dattype, 'int')
            dna = uint64(sum(dna.*binbase, 2));
        end
    end
end
```

E. Fitness Function

The fitness of each individual is equal to it's functional value. When the mimnimum is to be found, the fitness is taken as the reciprocal of the functional value.

In the roulette selection, the fitness is converted into probability. To avoid unpleassantries with negative probabilities, the input data is shifted into positive numbers. An auxiliary perturbation factor $p_{\text{aux}} = 10^{-3}$ is added to the data to avoid division by zero in the minimum case.

```
function fitness = fitnessFunc(data)
    p_aux = 1e-3;

    data_shift = data - min(data) + p_aux;

    if strcmp(minmax, 'max)
        fitness = data_shift;
    else
        fitness = 1 / data_shift;
    end
end
```

III. Algorithm Performance for the Target Functions

Since the 1D function only has a single maximum, the algorithm[2] is able to find it without problems. The evolution of the population is illustrated in figure 2 as the position of the individuals across generations. The algorithm reaches relatively high levels of precision, as is illustrated by figure 3.

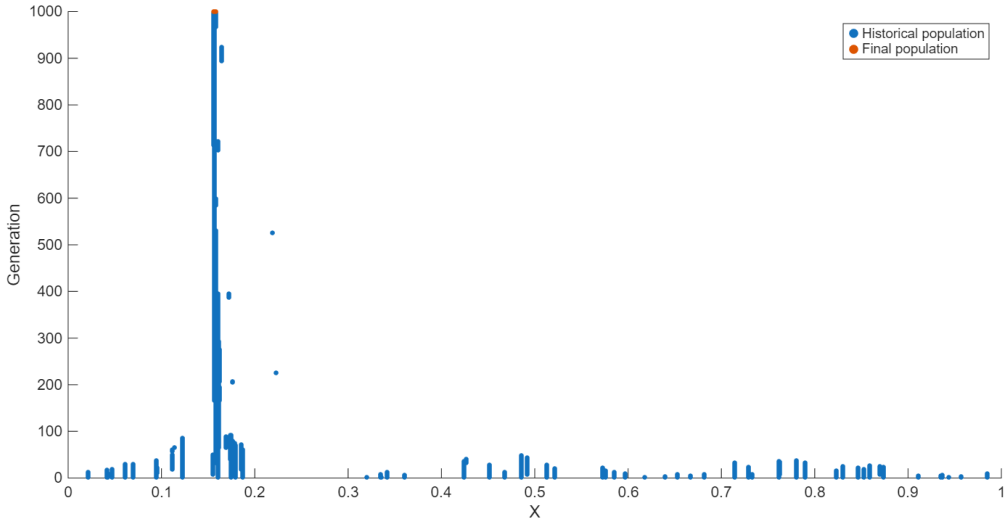


Fig. 2 Population evolution, 1D, maximum

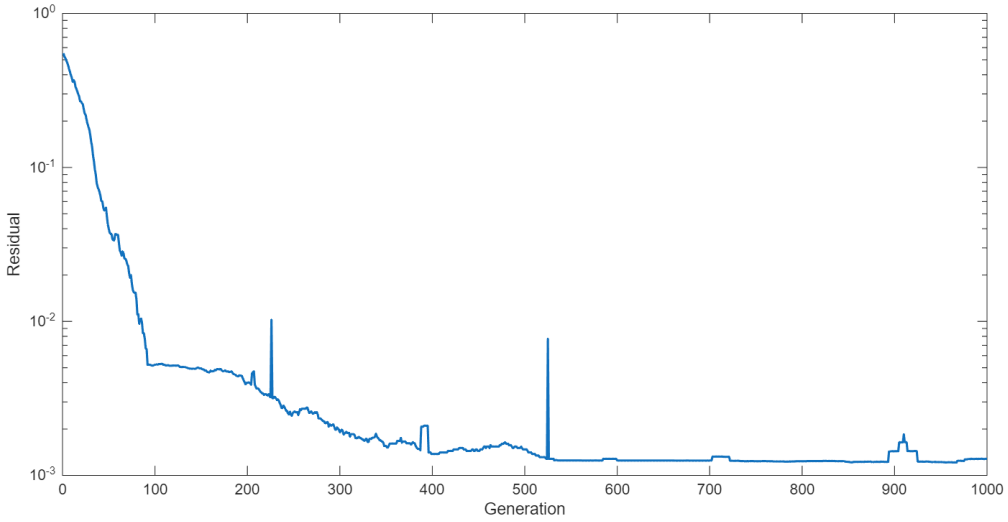


Fig. 3 Residual across generations, 1D, maximum

[2]The algorithm parameters used in the following are described in section V.

Finding the minima of the 1D function, however, is much more tricky, since there are infinitely many of them. Not even on a finite interval is the algorithm able to reliably find all of them. The reason is that the population converges towards one local maximum but witout a lucky mutation, it cannot escape.

In figure 4, population size was set to $N = 200$, as low values proved likely to converge to a single minimum. It is however ultimately impossible to find all minima reliably. It is apparent that because of the high population size, the precision will suffer (depicted in figure 5).

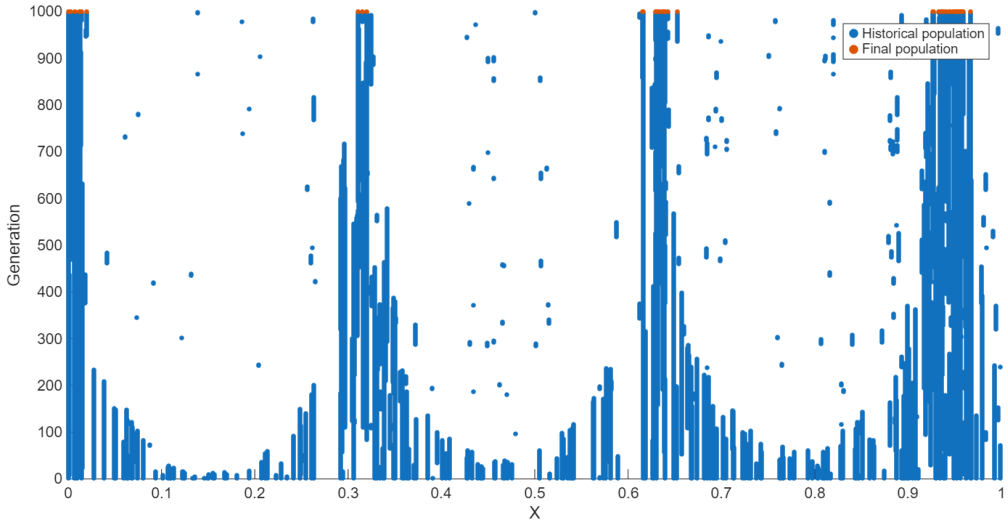


Fig. 4 Population evolution, 1D, minimum

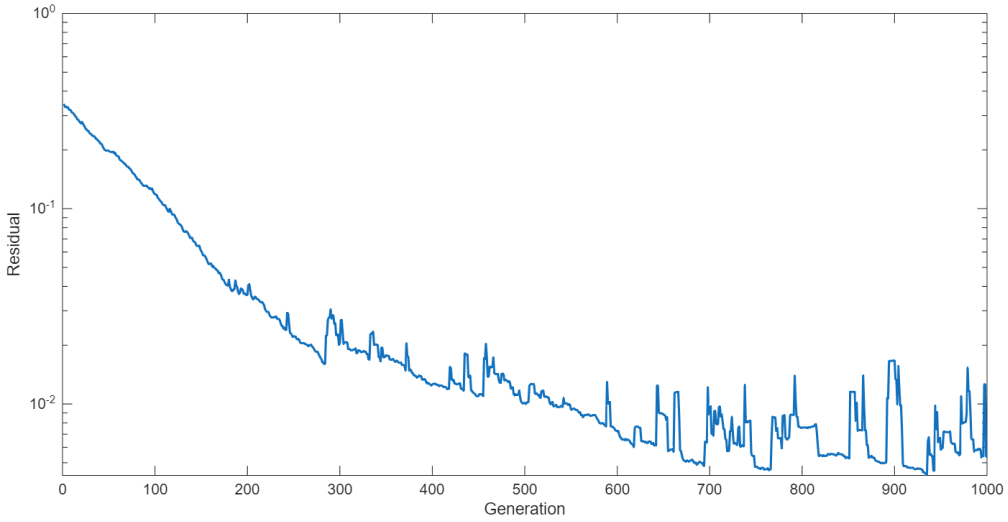


Fig. 5 Residual across generations, 1D, minimum

For the 2D function, the algorithm performs significantly worse. The convergence is much slower than in the 1D case, which results in the algorithm needing much more generations to reach a similar level of precision. Residuals for the minimum and maximum case are shown on figure 6.

The 2D case highligts a curious shortcoming of the algorithm: as the population converges to a small radius around a point, the crossover event ceases to be effective, as it becomes impossible to generate new individuals (since the parents are always near identical). The only way to escape this is through mutation.

Interestingly, higher population sizes seem to converge much more steadily, albeit the convergence speed is not always higher.

An alternative approach to battle the "clustering" phenomenon is to employ anti-clustering measures, for example temporarily raising the mutation probability. As the population clustered to a small region is unable to produce non-identical children, raising the mutation rate increases the chance to escape the area. The situation depicted in figure 7 is however rather lucky and overall, this approach is not very reliable.

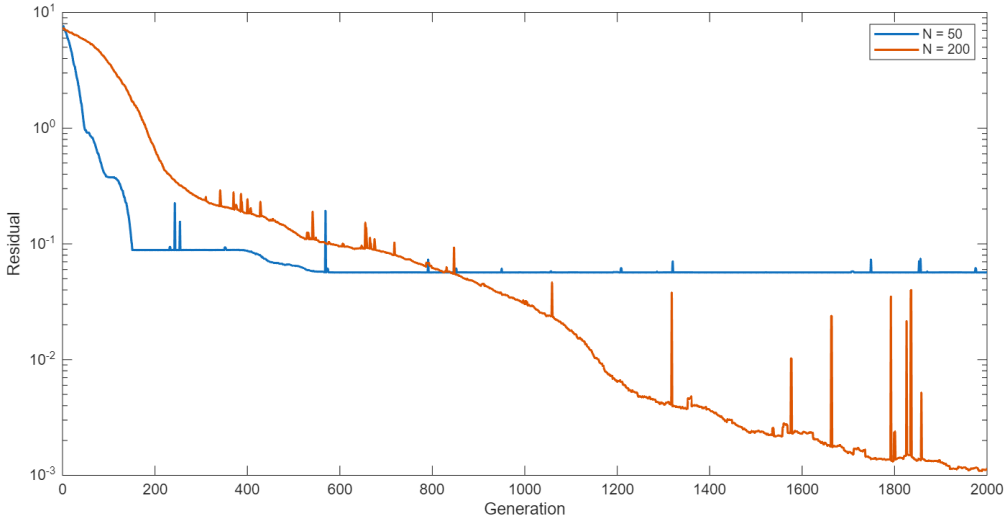


Fig. 6 Residual across generations, 2D, maximum

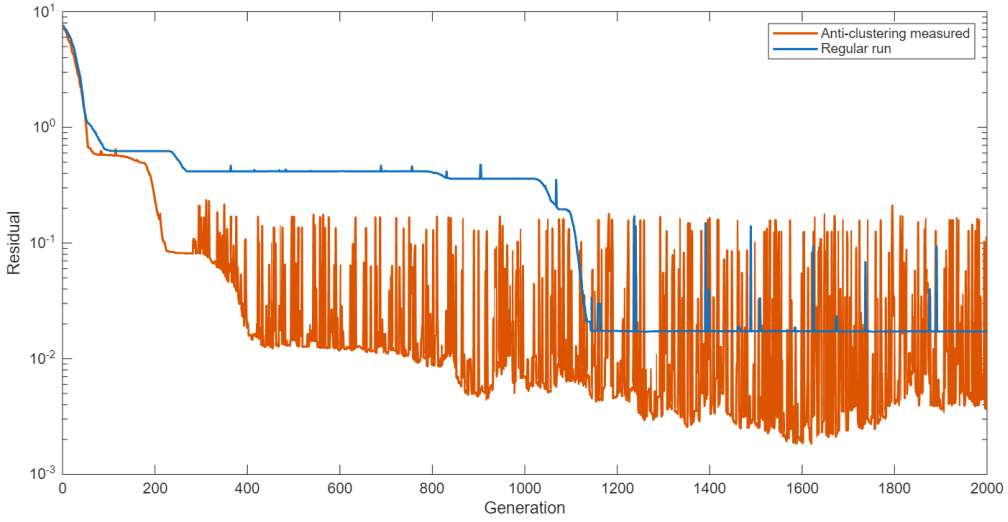


Fig. 7 Residual across generations with anti-clustering measures

IV. Effects of Evolution Parameters

A. Crossover Participants Selection Process

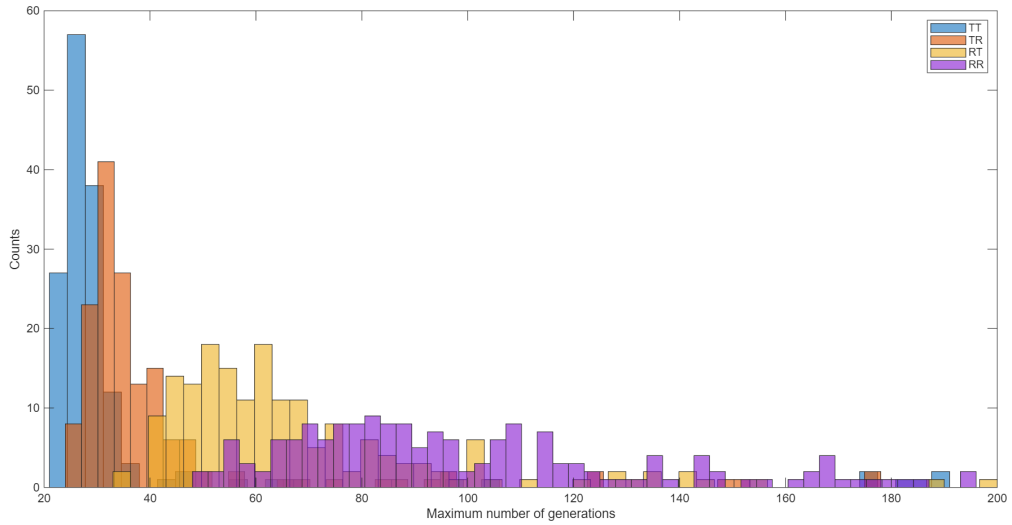
Two different strategies were implemented for selecting the participants of the crossover event. As was mentioned before, the algorithm chooses two parents among the best performing individuals and two death candidates among the worst performing.

The first strategy is a 'roulette selection', in which the selection probability is weighted by the indviduals' fitness. The second strategy is a 'tournament selection', for which a set number of participants is randomly selected. From those, only the two best (or worst, if selecting the death candidates) individuals are selected.

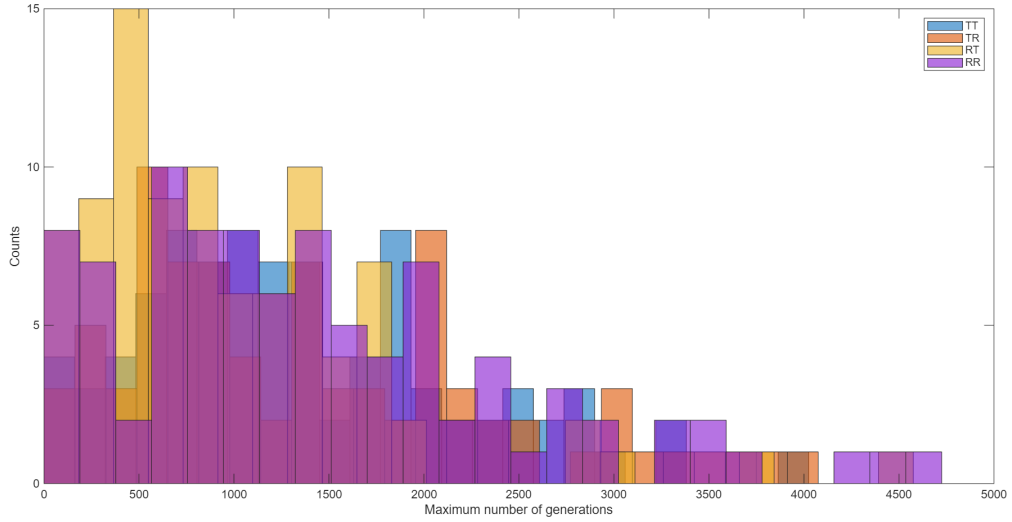
Figure 8 illustrates the effect of different combinations of the strategies on the number of generations needed to reach an error margin of 10^{-2}. The meaning of the legends is shown in table 1.

As is apparent from figure 8a, the strategy employing the tournament selection in both cases converges the fastest. This is likely due to always selecting the best (or worst) performing individuals from a large set. As was expected, the RR approach is the slowest, since this method is the most random.

Unfotunately, the choice of selection strategy seems to have no significant impact for the 2D function (see figure 8b).



(a) 1D function



(b) 2D function

Fig. 8 Comparison of the crossover participants selection strategies

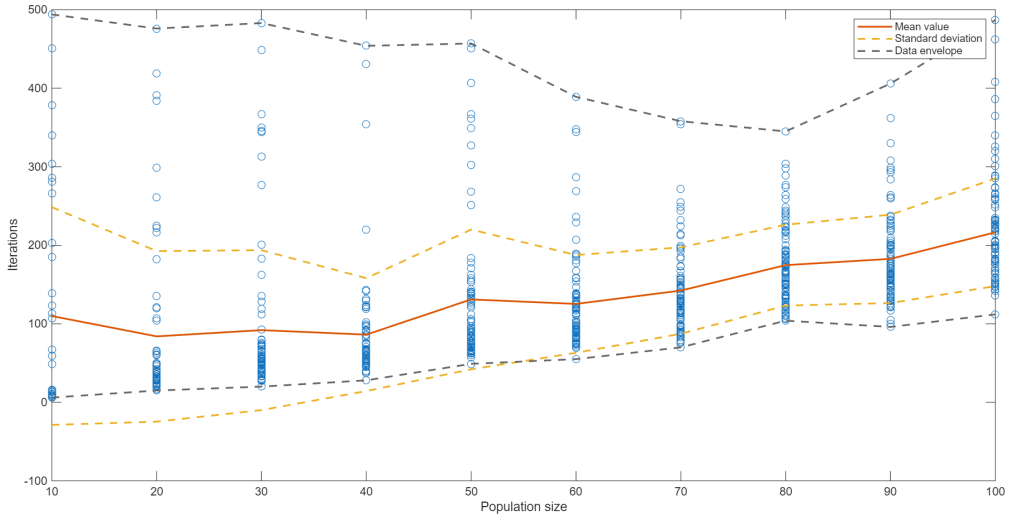
		Death candidate selection	
		Roulette	Tournament
Parent selection	Roulette	**RR**	**RT**
	Tournament	**TR**	**TT**

Table 1 Selection strategy legend

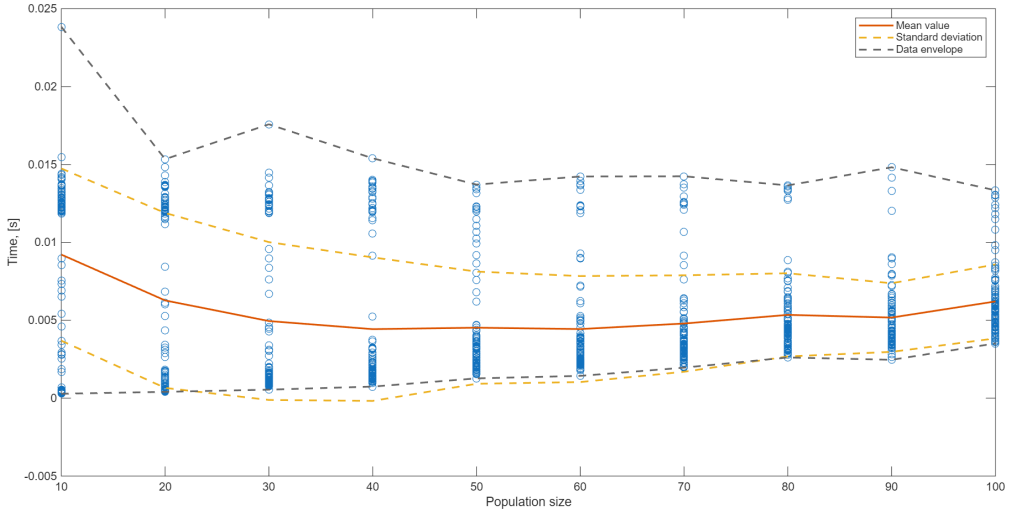
B. Population Size

Population size was swept from 10 to 100 with a step of 10. Figure 9a shows that larger populations tend to converge slower. For fast convergence, population size should be low, but not too low, as then the computation tome for each generation rises (see figure 9b).

Interestingly, when the maximum generations are capped, the result precision rises with the population size, while the computation time rises only slightly, as is shown on figures 10a and 10b.

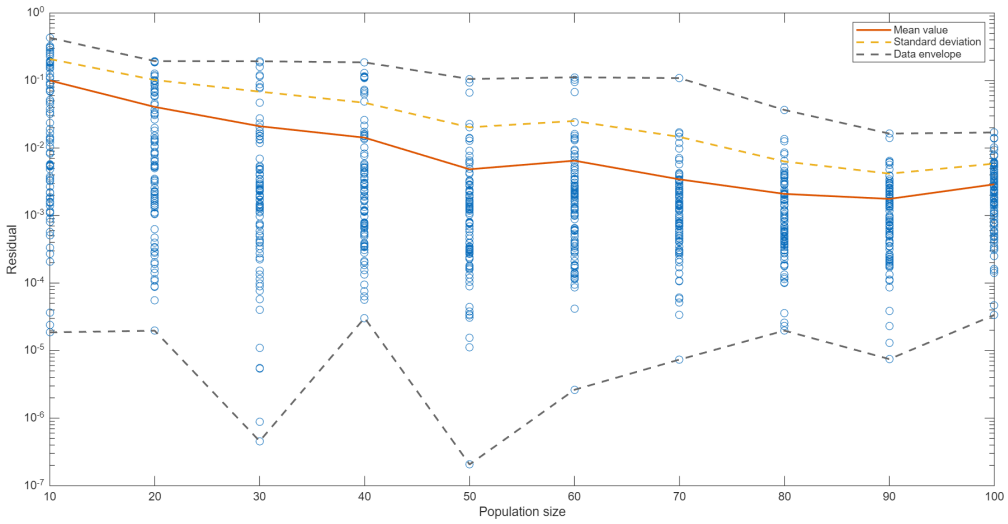


(a) Number of generations needed for convergence

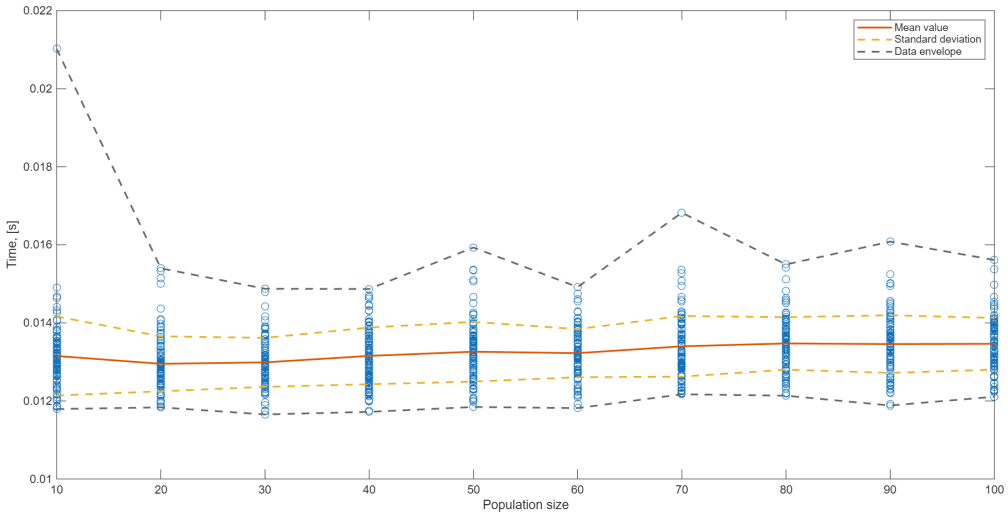


(b) Computation time of each generation

Fig. 9 Effect of population size, fixed error margin



(a) Result error

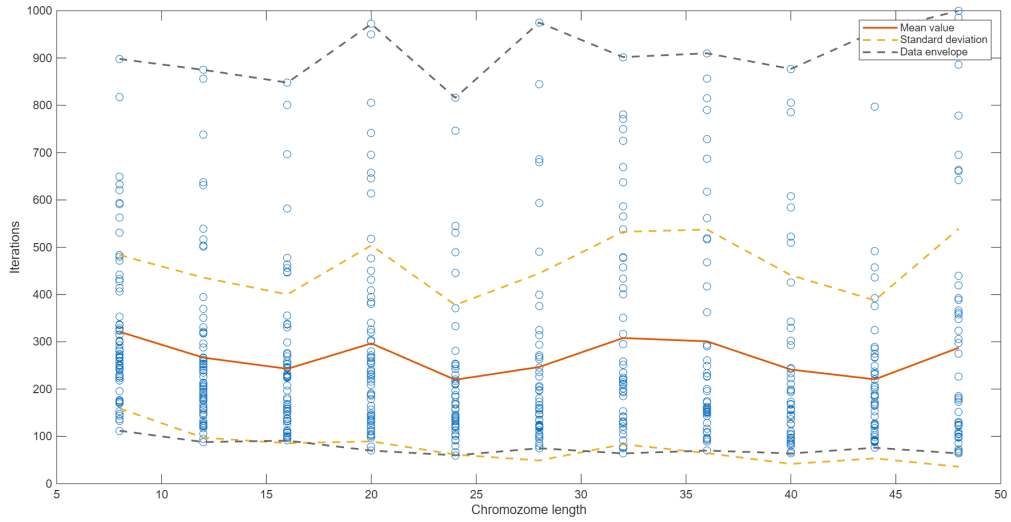


(b) Computation time of each generation

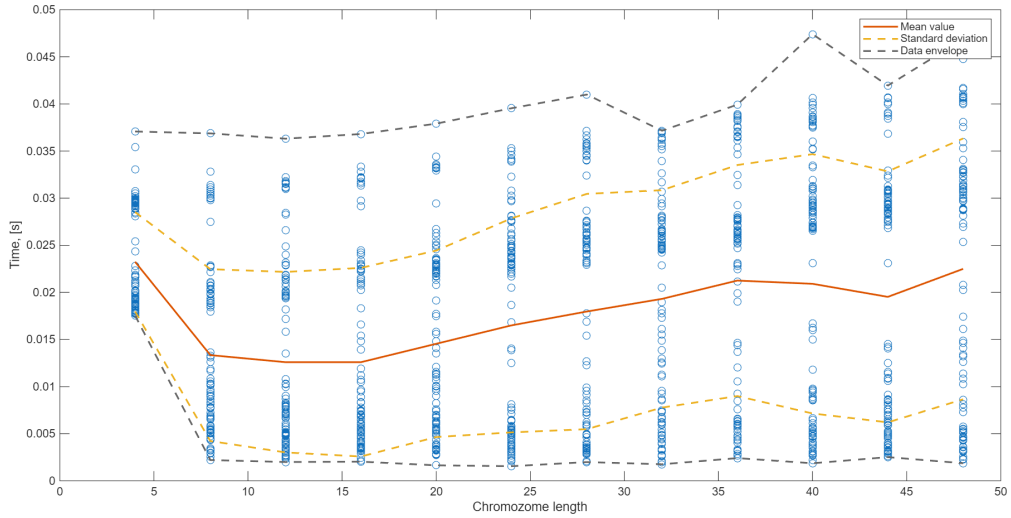
Fig. 10 Effect of population size, fixed number of generations

C. Chromozome Length

Chromozome length was swept from 4 to 48 with a step of 2. It's effect on the number of generations needed for convergence in figure 11a can perhaps be described as a slight decreasing trend. In every case, the impact it has on the computation time is quite severe (figures 11b and 12b).

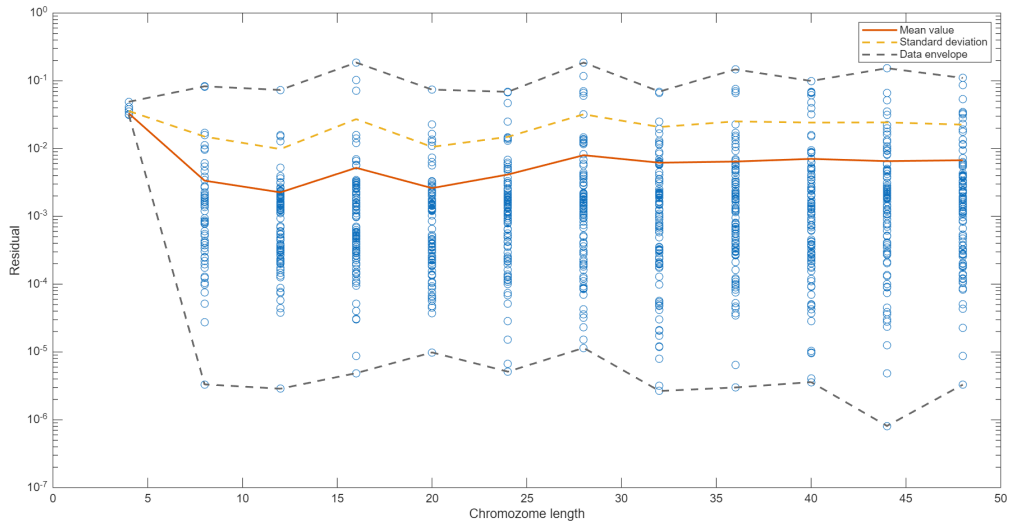


(a) Number of generations needed for convergence

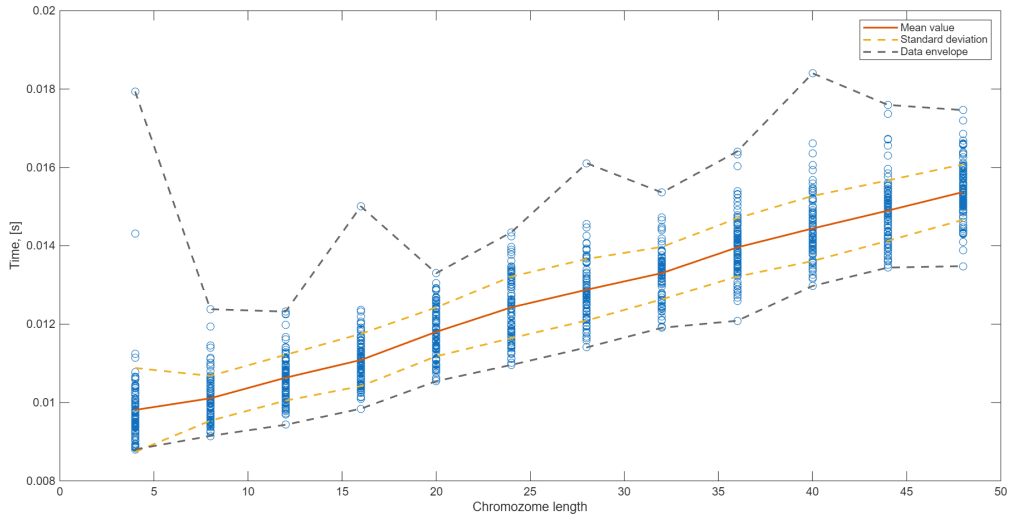


(b) Computation time of each generation

Fig. 11 Effect of chromozome length, fixed error margin



(a) Result error

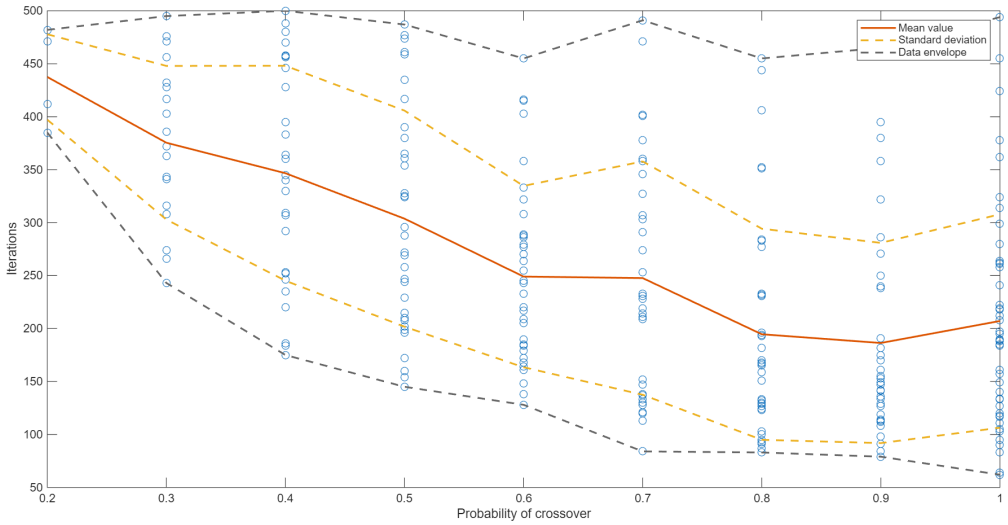


(b) Computation time of each generation

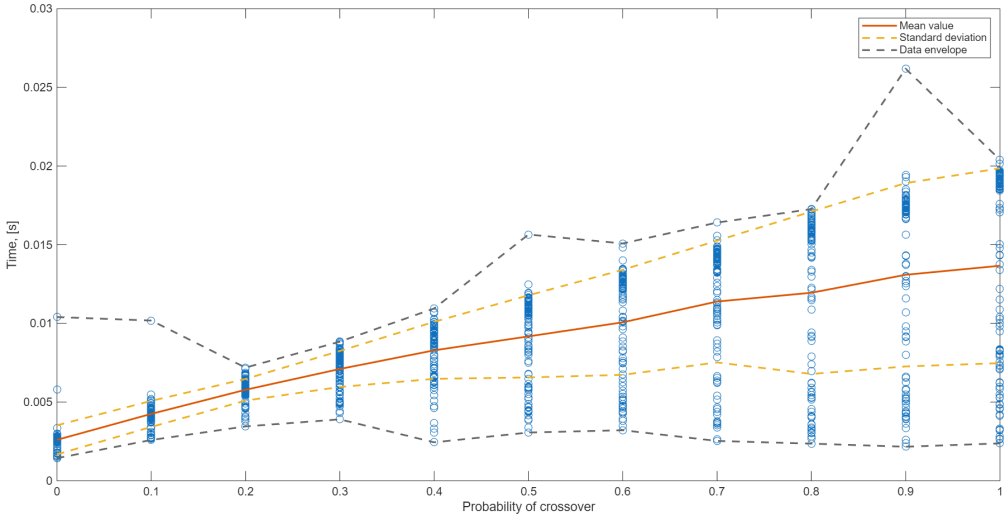
Fig. 12 Effect of chromozome length, fixed number of generations

D. Probability of Crossover

Crossover probability was swept from 0 to 1 with a step of 0.1. It is immediately obvious from figures 13a and 14a that a high probability yields precise results fast. The high computation time caused by high probabilities is far outweighed by their performance benefits.

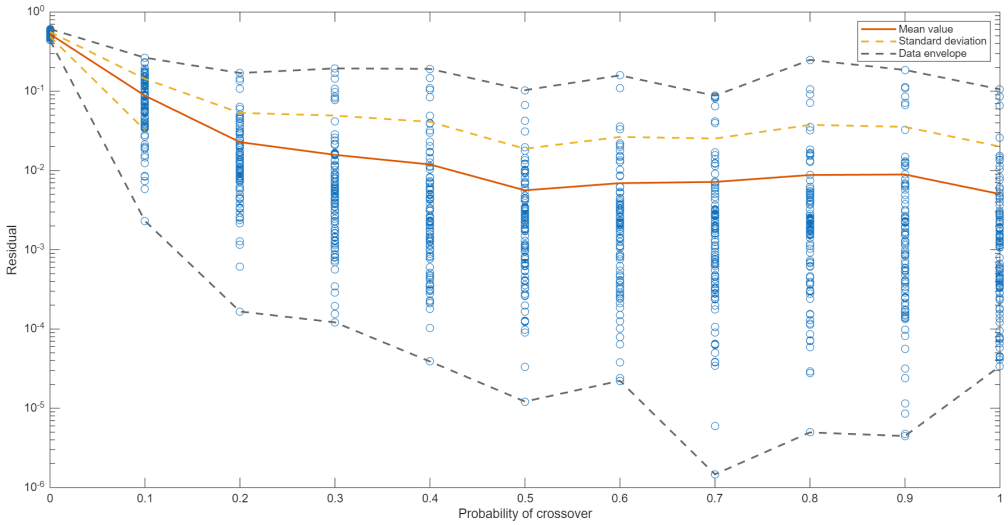


(a) Number of generations needed for convergence

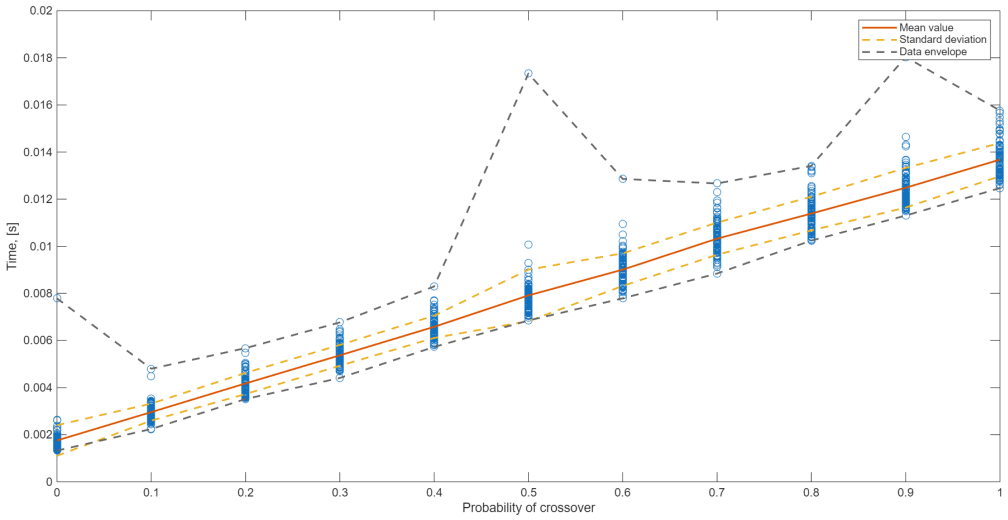


(b) Computation time of each generation

Fig. 13 Effect of crossover probability, fixed error margin



(a) Result error

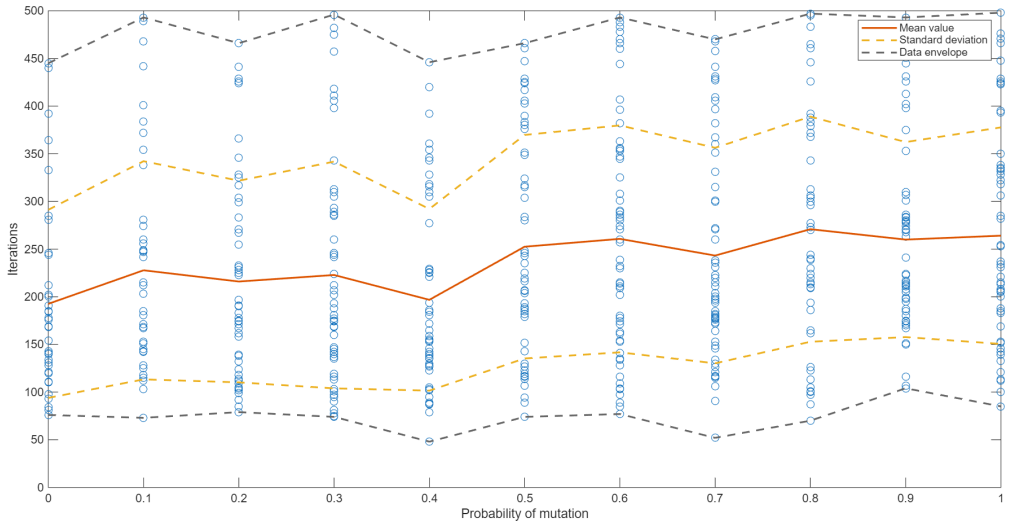


(b) Computation time of each generation

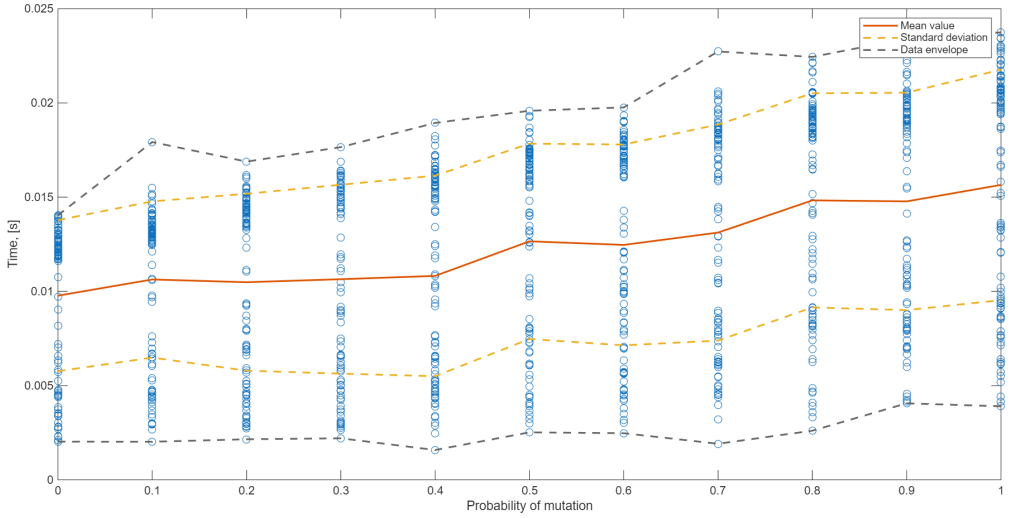
Fig. 14 Effect of crossover probability, fixed number of generations

E. Probability of Mutation

The probability of mutation was swept in the same range as the crossover probability. For the convergence speed, figure 15a shows an opposite trend as the crossover probability, indicating that low mutation probabilities should be used. The result residual is not particularly affected (see figure 16a). The computation time behaves the same as with crossover probability.

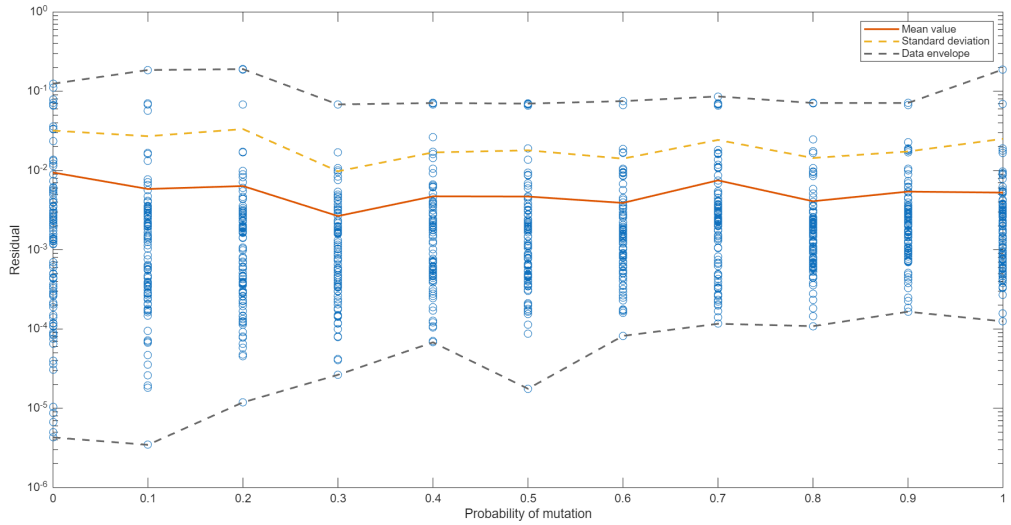


(a) Number of generations needed for convergence

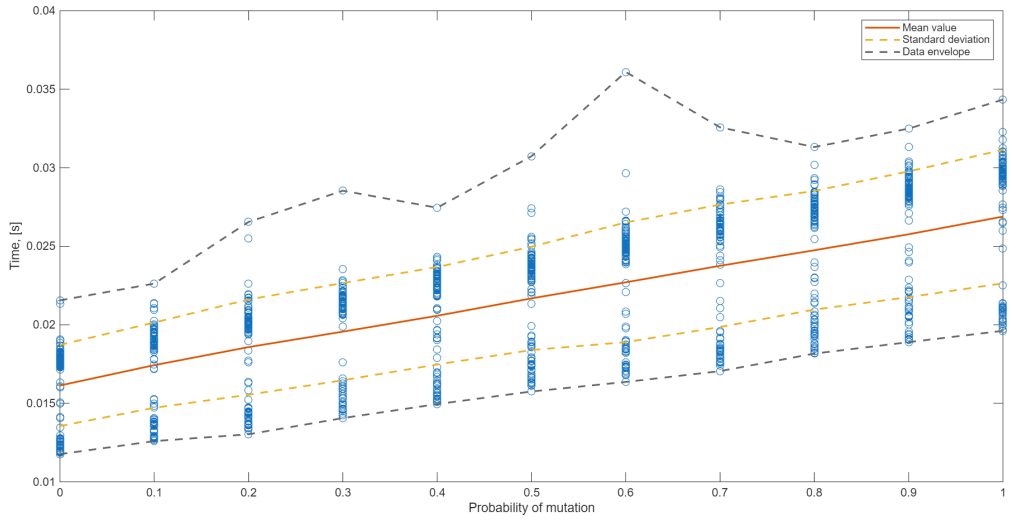


(b) Computation time of each generation

Fig. 15 Effect of mutation probability, fixed error margin



(a) Result error

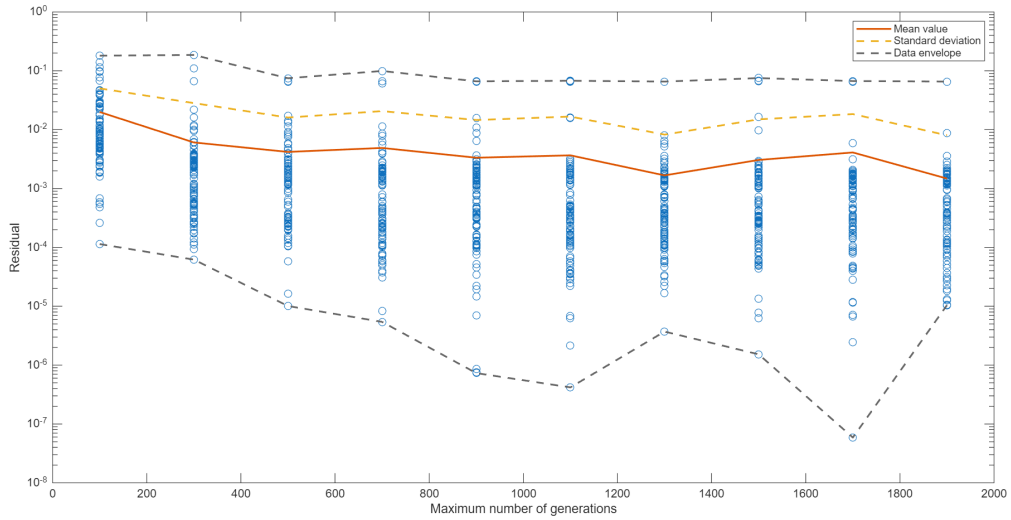


(b) Computation time of each generation

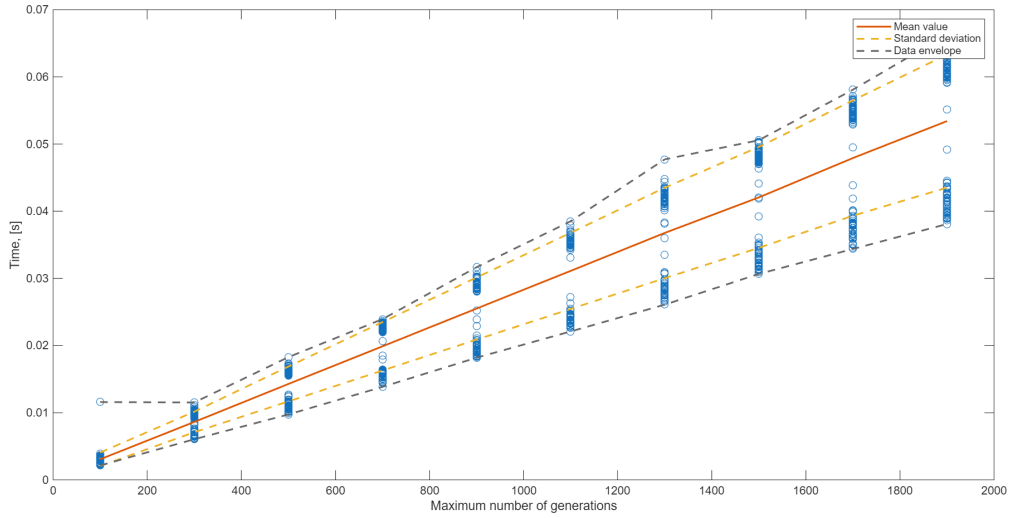
Fig. 16 Effect of mutation probability, fixed number of generations

F. Maximum Number of Generations

Generation limit was swept from 100 to 2000 with a step of 200. The result precision gets higher with the numbe rof generaions, but the trend is surprisingly slow (see figure 17a). This may hint at a flawed implementation or a fundamental shortcoming of the algorithm itself. Contrary to expectations, the computation time for each generations in figure 17b is quite constant. This is likely not a true representation of the time needed to compute one generation. Likely, it is artificially lengthened by handling increasingly large arrays of data containing results of past generations.



(a) Result error



(b) Computation time of each generation

Fig. 17 Effect of the number of generations

G. Parameter sweeps with the 2D function

Unfortunately, the algorithm doesn't preform quite to satisfaction in the 2D case. The results are distorted by severe outliers (in terms of residual), as is apparent from figures 18 and 19. This fact renders any conculsions regarding effets of the parameters untrustworthy.

It can be however expected that the alorithm should qualitatively behave in the same way as in the 1D case.

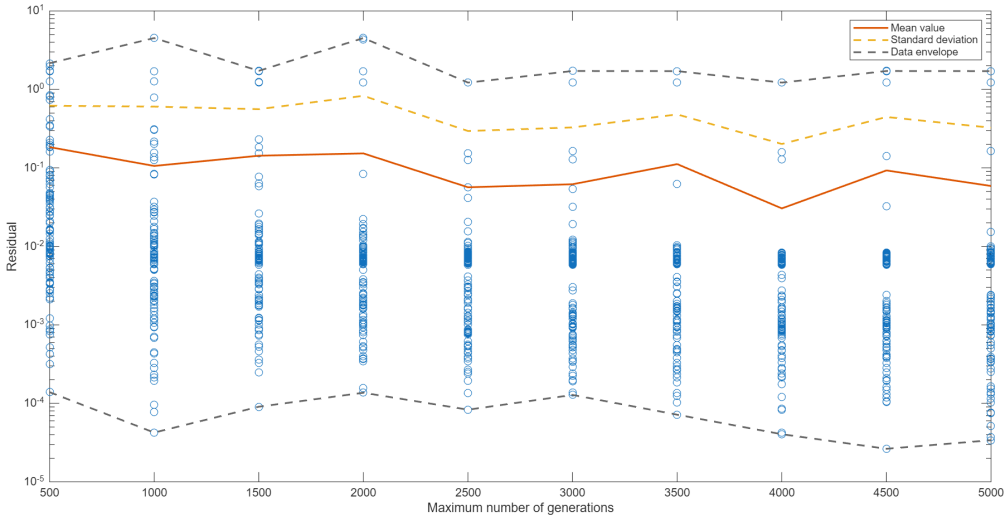


Fig. 18 Effect of the number of generations, 2D function

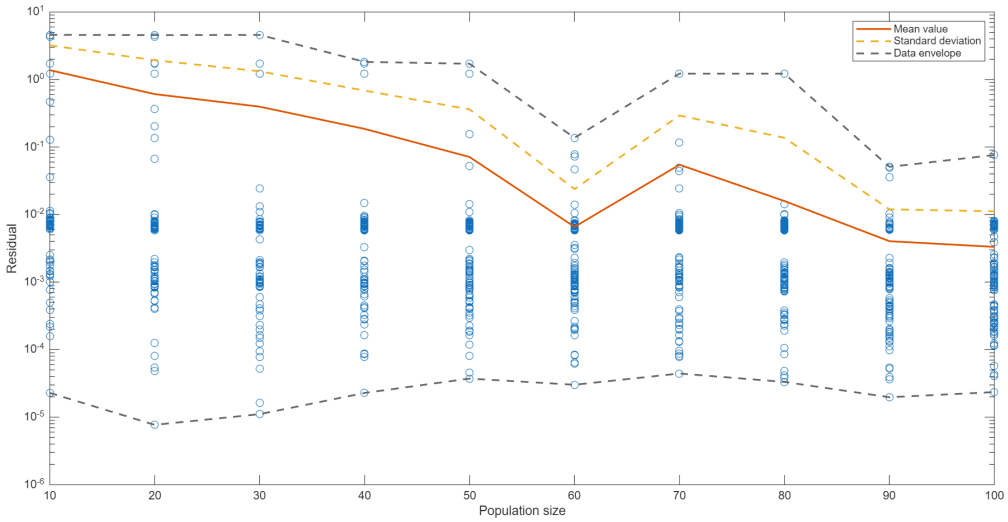


Fig. 19 Effect of population size, 2D function

V. Conclusions

The functionality of the algorithm was proven on two functions. While the algorithm performed to satisfaction on the 1D function, it struggled with the 2D function (relative to it's performance in the previous case). Thereafter, several parameter sweeps were performed as well as an investigation into strategies for selection of crossover participants.

To keep to the algorithm's original purpose, the 'RR' strategy (employing the roulette selection for selecting both the parents and the death candidates) should be used. However, the 'TT' strategy (employing the tournament selection) can significantly hasten the algorithm's convergence.

According to the parameter sweep, high crossover probability and low mutation probability should be set. The population size should be relatively low (i.e. 40). The length of the chromozome should be set as high as possible, albeit it's effect is much less significant than the other parameters'.

Appendix: Algorithm Parameters Used for Results

In section III, performance of the algorithm was shown without explicitly stating the used parameters.

Unless stated otherwise (i.e. when investigating the parameter sweeps or when raising population sizes to find minima), the algorithm used parameters further described in table 2. The $error\_margin$ and i_{max} parameters are mutually exclusive.

The default strategy for selecting the crossover participants was the roulette selection. The tournament pool size parameter ($tpoolsz$) is naturally only used in the tournament selection.

Population size	N	50
Chromozome length	$chrlen$	32
Probability of crossover	P_{c}	0.95
Probability of mutation	P_{m}	0.05
Tournament pool size	$tpoolsz$	$\lceil \frac{N}{2} \rceil$
Target error margin	$error\_margin$	10^{-3}
Mxaximum number of generations	$i_{\max}$	500

Table 2 Default parameters of the algorithm